LECTURE SERIES · DATABASE SYSTEMS · WEEK 5

Procedures, Triggers & Virtual Tables

From Reusable Server-Side Logic to Virtual & Materialized Tables

Md. Rakib Trofder & Saad Ahmed

Lecturers, Department of CSE, Bangladesh Army University of Science & Technology

Stored Procedures · Functions · Triggers · Views · Materialized Views

TODAY'S ROADMAP

Week 5 Agenda

We move from one-off SQL statements to programmable, reusable database objects — code that lives and runs inside the database itself.

01

Stored Procedures

Named, callable blocks of SQL logic

02

Functions

Routines that return a usable value

03

Triggers

Logic that fires automatically on events

04

Views

Virtual tables from a stored query

05

Materialized Views

Views with a physically stored result

06

Lab Task

Hands-on library-system exercise

REUSABLE, NAMED SQL LOGIC

Stored Procedures

A stored procedure is a precompiled, named block of SQL saved inside the database. It can take IN / OUT parameters and run several statements, but it is invoked with CALL — it does not plug into an expression the way a function does.

Example 1 — Insert with parameters

Adds a new student record to the table.

```
CREATE PROCEDURE AddStudent(
  p_name VARCHAR(50),
  p_dept VARCHAR(20))
LANGUAGE plpgsql
AS $$
BEGIN
  INSERT INTO Student(name, dept)
  VALUES (p_name, p_dept);
END; $$;

CALL AddStudent('Rafi','CSE');
```

Example 2 — Return via OUT parameter

Counts rows and passes the result back to the caller.

```
CREATE PROCEDURE GetStudentCount(
  INOUT total INT)
LANGUAGE plpgsql
AS $$
BEGIN
  SELECT COUNT(*) INTO total
  FROM Student;
END; $$;

CALL GetStudentCount(0);
```

ROUTINES THAT RETURN A VALUE

Functions

A function is a named routine that MUST return exactly one value. Because of that, it can be used directly inside a SELECT list, a WHERE clause, or any expression — a procedure cannot.

Example 1 — Scalar value from a column

Computes a student's age from their date of birth.

```
CREATE FUNCTION GetAge(dob DATE)
RETURNS INT AS $$
BEGIN
  RETURN DATE_PART('year', AGE(dob));
END; $$ LANGUAGE plpgsql;

SELECT name, GetAge(dob)
FROM Student;
```

Example 2 — Used inside an expression

Converts numeric marks into a letter grade.

```
CREATE FUNCTION GetGrade(m INT)
RETURNS CHAR(2) AS $$
BEGIN
  RETURN CASE
    WHEN m >= 80 THEN 'A+'
    WHEN m >= 60 THEN 'A'
    ELSE 'F' END;
END; $$ LANGUAGE plpgsql;

SELECT id, GetGrade(marks)
FROM Result;
```

LOGIC THAT FIRES ON AN EVENT

Triggers

A trigger is a stored program bound to a table that fires automatically BEFORE or AFTER an INSERT, UPDATE, or DELETE. No one ever calls it explicitly — the database event itself is the trigger.

Example 1 — BEFORE INSERT

Stamps every new row with its creation time.

```
CREATE FUNCTION set_created_at()
RETURNS TRIGGER AS $$
BEGIN
  NEW.created_at := now();
  RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_set_created
BEFORE INSERT ON Student
FOR EACH ROW
EXECUTE FUNCTION set_created_at();
```

Example 2 — AFTER UPDATE

Writes an audit-log row whenever a salary changes.

```
CREATE FUNCTION log_salary_change()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO Salary_Audit(emp_id,
    old_sal, new_sal)
  VALUES (OLD.id, OLD.salary, NEW.salary);
  RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_log_salary
AFTER UPDATE ON Employee
FOR EACH ROW
EXECUTE FUNCTION log_salary_change();
```

A VIRTUAL TABLE

Views

A view is a virtual table defined by a stored SELECT query. It stores no data of its own — every time it is queried, the underlying base tables are re-evaluated live.

Example 1 — Filtering rows

A reusable, simplified window onto one table.

```
CREATE VIEW CSE_Students AS
SELECT id, name, cgpa
FROM Student
WHERE dept = 'CSE';

SELECT * FROM CSE_Students
WHERE cgpa > 3.5;
```

Example 2 — Joining tables

Hides a multi-table join behind one simple name.

```
CREATE VIEW Student_Results AS
SELECT s.name, r.course_id,
       r.marks
FROM Student s
JOIN Result r
  ON s.id = r.student_id;
```

A PHYSICALLY STORED RESULT

Materialized Views

Like a view, but its result set is physically stored on disk at creation time. It trades live accuracy for speed, so it must be REFRESHed on a schedule to reflect base-table changes.

Example 1 — Precomputed aggregate

Stores each department's average CGPA (PostgreSQL).

```
CREATE MATERIALIZED VIEW
  Dept_Avg_CGPA AS
SELECT dept, AVG(cgpa) avg_cgpa
FROM Student
GROUP BY dept;

REFRESH MATERIALIZED VIEW
  Dept_Avg_CGPA;
```

Example 2 — Refresh for a dashboard

Keeps a sales summary fast without recomputing.

```
CREATE MATERIALIZED VIEW
  Monthly_Sales AS
SELECT product_id,
       SUM(qty) total_qty
FROM Orders
GROUP BY product_id;

REFRESH MATERIALIZED VIEW
  CONCURRENTLY Monthly_Sales;
```

CHOOSING THE RIGHT ONE

Views vs. Materialized Views

Both hide a query behind a name. The difference is whether that query runs every time, or only when you tell it to.

Attribute	View	Materialized View
Storage	No physical storage — virtual only	Physically stores the result set
Data freshness	Always live and current	A snapshot — can be stale
Query speed	Same cost as the underlying query	Fast — reads precomputed data
Refresh needed?	No — recomputed every query	Yes — manual or scheduled REFRESH
Best for	Simplifying joins, access control	Expensive aggregates, dashboards

LAB TASK · WEEK 5

Library Management System

`Book(book_id, title, author, copies_available)` · `Member(member_id, name)` · `Loan(loan_id, book_id, member_id, loan_date, return_date, fine)`

1. **Stored Procedure** — IssueBook(p_book_id, p_member_id) — inserts a Loan row and decrements copies_available, only if a copy is available.

2. **Function** — CalculateFine(p_loan_id) — returns the fine at 5 Tk/day for every day overdue past a 14-day limit.

3. **Trigger** — Prevent a new Loan from being inserted when a book's copies_available is already 0.

4. **View** — Active_Loans — shows every loan with return_date IS NULL, joined with book title and member name.

5. **Materialized View** — Popular_Books — stores each book's total loan count; include the REFRESH statement.

6. **Compare** — Query Popular_Books against an equivalent live view and note the difference in speed.